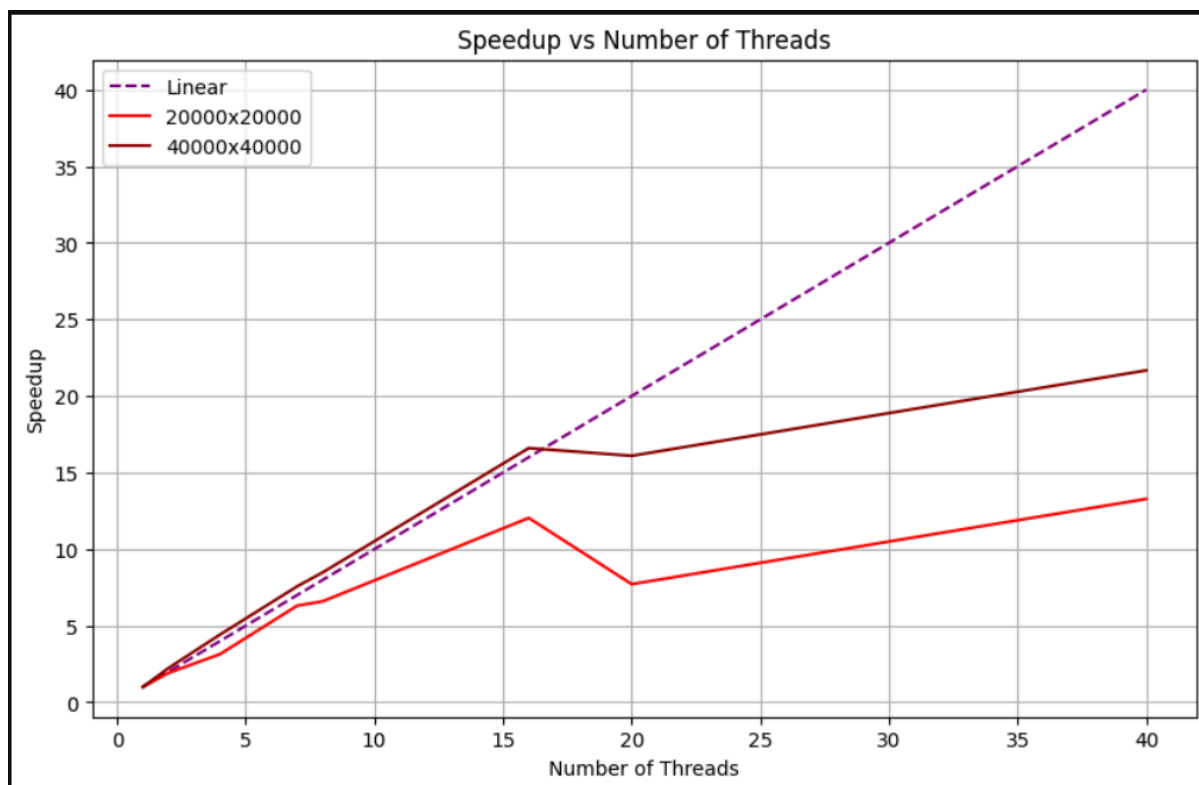


Вывод к заданию 3 (1 задача):



Основные замечания:

1. Общее поведение потоков

Программа делит работу (инициализацию матрицы, вектора и вычисление произведения) между потоками, распределяя строки матрицы поровну. Каждый поток выполняет свою часть независимо, что теоретически должно давать линейное ускорение при увеличении числа потоков. Но, на деле, мы можем наблюдать отклонения от линейного роста из-за накладных расходов и ограничений системы.

Рассмотрим детально наш график:

- **До 8 потоков:**
 - Для обеих матриц ускорение растет почти линейно до 7–8 потоков. Например, для 20000x20000 ускорение на 7 потоках достигает 6.31, а для 40000x40000 — 7.57. Это близко к идеальному значению (7), что говорит о хорошей масштабируемости на малом числе потоков.

- Для 40000x40000 ускорение даже немного превышает идеальное на 8 потоках (8.49 против 8), что может быть связано с кэш-эффектами или оптимизацией процессора.
- **После 8 потоков:**
 - На 16 потоках ускорение продолжает расти: 12.04 для 20000x20000 и 16.60 для 40000x40000. Для большей матрицы ускорение ближе к линейному (16.60 против 16), что показывает, что увеличение объема данных позволяет лучше использовать потоки.
 - Однако на 20 потоках наблюдается просадка: 7.71 для 20000x20000 (падение с 12.04) и 16.10 для 40000x40000 (небольшое снижение с 16.60). Это явный признак перегрузки системы.
 - На 40 потоках ускорение снова растет: 13.28 для 20000x20000 и 21.67 для 40000x40000, но все еще далеко от линейного (40).

2. Факторы, влияющие на ускорение:

- **Объем данных:**

Матрица 40000x40000 (в 4 раза больше данных, чем 20000x20000) показывает лучшее ускорение на больших числах потоков (например, 21.67 против 13.28 на 40 потоках). Это связано, возможно с тем, что при большем объеме вычислений накладные расходы на управление потоками становятся менее значительными по сравнению с полезной работой. Для 20000x20000 накладные расходы начинают доминировать раньше, что приводит к более выраженным просадкам (например, на 20 потоках).

- **Накладные расходы на потоки:**

Создание, синхронизация и завершение потоков требуют времени. На малом числе потоков (до 8) эти расходы минимальны, но с ростом числа потоков (16, 20, 40) они становятся заметными. На 20 потоках просадка особенно заметна для 20000x20000, так как система, вероятно, достигает предела по количеству эффективно работающих потоков.

- **Конкуренция за ресурсы:**

Память: матрица хранится как одномерный вектор (`std::vector<double>`), и доступ к ней из разных потоков может вызывать конкуренцию за кэш процессора, особенно при большом числе потоков. Процессор: к примеру, если мы рассматриваем работу нашей программы на 8 физических ядрах (или 16 с гиперпоточностью), то 20 или 40 потоков приводят к

переключению контекста, что снижает производительность. Это объясняет просадку на 20 потоках.

- **Инициализация массивов:**

Инициализация матрицы и вектора также выполняется параллельно, но это относительно легкая операция по сравнению с умножением. Для 20000x20000 инициализация занимает меньше времени, и основное влияние на ускорение оказывает само умножение.

3. Общее сравнение матриц 20000x20000 и 40000x40000:

- Матрица 40000x40000 масштабируется лучше, особенно на большом числе потоков. Это объясняется тем, что больший объем вычислений (в 4 раза больше элементов) позволяет лучше распределить работу между потоками, минимизируя относительное влияние накладных расходов.
- Для 20000x20000 просадки более выражены, так как объем данных меньше, и накладные расходы (создание потоков, синхронизация) становятся более значительными по сравнению с полезной работой.

4. Общие выводы:

- **Масштабируемость:**
 - Программа демонстрирует хорошую масштабируемость до 8–16 потоков, особенно для матрицы 40000x40000, где ускорение на 16 потоках достигает 16.60 (почти линейное). Для 20000x20000 максимальное ускорение (12.04 на 16 потоках) также неплохое, но дальше начинаются просадки.
 - На большом числе потоков (20, 40) масштабируемость ухудшается из-за конкуренции за ресурсы и накладных расходов на управление потоками.
- **Влияние объема данных:**
 - Большой объем данных (40000x40000) позволяет лучше использовать параллелизм, так как вычисления занимают больше времени, и накладные расходы становятся менее значительными.
 - Для меньшей матрицы (20000x20000) накладные расходы начинают доминировать раньше, что приводит к более выраженным просадкам.